



Final Project - Impact of AI on Students

This is a submission to the final project to the course Supervised Machine Learning: Classification by IBM on Coursera.

```
In [1]: #The following packages need to be installed:  
!pip install kagglehub[pandas-datasets]  
!pip install scikit-learn  
!pip install pandas  
!pip install numpy  
!pip install matplotlib  
!pip install seaborn
```

Requirement already satisfied: kagglehub[pandas-datasets] in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (1.0.1)

Requirement already satisfied: kagglesdk<1.0,>=0.1.22 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from kagglehub[pandas-datasets]) (0.1.23)

Requirement already satisfied: packaging in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from kagglehub[pandas-datasets]) (26.2)

Requirement already satisfied: pyyaml in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from kagglehub[pandas-datasets]) (6.0.3)

Requirement already satisfied: requests in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from kagglehub[pandas-datasets]) (2.34.2)

Requirement already satisfied: tqdm in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from kagglehub[pandas-datasets]) (4.67.3)

Requirement already satisfied: pandas in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from kagglehub[pandas-datasets]) (3.0.3)

Requirement already satisfied: protobuf in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from kagglesdk<1.0,>=0.1.22->kagglehub[pandas-datasets]) (7.34.1)

Requirement already satisfied: numpy>=1.26.0 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from pandas->kagglehub[pandas-datasets]) (2.4.5)

Requirement already satisfied: python-dateutil>=2.8.2 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from pandas->kagglehub[pandas-datasets]) (2.9.0.post0)

Requirement already satisfied: charset_normalizer<4,>=2 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from requests->kagglehub[pandas-datasets]) (3.4.7)

Requirement already satisfied: idna<4,>=2.5 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from requests->kagglehub[pandas-datasets]) (3.15)

Requirement already satisfied: urllib3<3,>=1.26 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from requests->kagglehub[pandas-datasets]) (2.7.0)

Requirement already satisfied: certifi>=2023.5.7 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from requests->kagglehub[pandas-datasets]) (2026.4.22)

Requirement already satisfied: six>=1.5 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from python-dateutil>=2.8.2->pandas->kagglehub[pandas-datasets]) (1.17.0)

Requirement already satisfied: scikit-learn in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (1.8.0)

Requirement already satisfied: numpy>=1.24.1 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from scikit-learn) (2.4.5)

Requirement already satisfied: scipy>=1.10.0 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from scikit-learn) (1.17.1)

Requirement already satisfied: joblib>=1.3.0 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from scikit-learn) (1.5.3)

Requirement already satisfied: threadpoolctl>=3.2.0 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from scikit-learn) (3.6.0)

Requirement already satisfied: pandas in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (3.0.3)

Requirement already satisfied: numpy>=1.26.0 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from pandas) (2.4.5)

Requirement already satisfied: python-dateutil>=2.8.2 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from pandas) (2.9.0.post0)

s/Code/.venv/lib/python3.12/site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Requirement already satisfied: numpy in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (2.4.5)
Requirement already satisfied: matplotlib in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (3.10.9)
Requirement already satisfied: contourpy>=1.0.1 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from matplotlib) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from matplotlib) (1.5.0)
Requirement already satisfied: numpy>=1.23 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from matplotlib) (2.4.5)
Requirement already satisfied: packaging>=20.0 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from matplotlib) (26.2)
Requirement already satisfied: pillow>=8 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from matplotlib) (12.2.0)
Requirement already satisfied: pyparsing>=3 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Requirement already satisfied: seaborn in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (0.13.2)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from seaborn) (2.4.5)
Requirement already satisfied: pandas>=1.2 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from seaborn) (3.0.3)
Requirement already satisfied: matplotlib!=3.6.1,>=3.4 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from seaborn) (3.10.9)
Requirement already satisfied: contourpy>=1.0.1 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (4.63.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (1.5.0)
Requirement already satisfied: packaging>=20.0 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (26.2)
Requirement already satisfied: pillow>=8 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (12.2.0)
Requirement already satisfied: pyparsing>=3 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (3.3.2)

Requirement already satisfied: python-dateutil>=2.7 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn) (2.9.0.post0)

Requirement already satisfied: six>=1.5 in /home/reinder/Documents/Code/.venv/lib/python3.12/site-packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)

```
In [2]: #For data handling and processing
import kagglehub
import pandas as pd
from sklearn.preprocessing import OrdinalEncoder, LabelEncoder, StandardScaler

#For plotting
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
sns.set_style('dark')
sns.set_theme('notebook')
sns.set_context('notebook')

#model_selection
from sklearn.model_selection import StratifiedShuffleSplit, GridSearchCV

#models
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.ensemble import VotingClassifier

#metrics
from sklearn.metrics import accuracy_score, precision_recall_fscore_support
```

```
/home/reinder/Documents/Code/.venv/lib/python3.12/site-packages/tqdm/auto.py:2
1: TqdmWarning: IPProgress not found. Please update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
from .autonotebook import tqdm as notebook_tqdm
```

Dataset Selection and Description

The dataset used in this notebook is the dataset "Impact of AI on Students" on Kaggle. As someone who recently graduated from university, I'm curious to see how AI usage affects current students. In particular, I will be studying the impact of AI on the risk of burnout among students.

The goal of this notebook is to determine whether or not AI-usage has a significant impact on risk of burnout among students.

Source

This data is publicly available on Kaggle. It can be found on the Kaggle website [here](#). We load it into a pandas dataframe.

```
In [3]: # Import the latest version of the data into a pandas DataFrame
kagglehub.dataset_download("laveshjadon/ai-impact-on-students")

df = kagglehub.dataset_load(
    kagglehub.KaggleDatasetAdapter.PANDAS,
    "laveshjadon/ai-impact-on-students",
    'ai_student_impact_dataset (1).csv'
)
df.head()
```

```
Out[3]:
```

	Student_ID	Major_Category	Year_of_Study	Pre_Semester_GPA	Weekly_GenA
0	100001	Humanities	Senior	2.418	
1	100002	Medical	Junior	3.821	
2	100003	Business	Freshman	3.398	
3	100004	Business	Senior	3.789	
4	100005	STEM	Sophomore	3.635	

Key Features and the Target

We are interested in the features related to AI. Those are the following:

Feature Name	Description
Weekly_GenAI_Hours	Average hours per week spent using Generative AI tools
Primary_Use_Case	Main purpose for which the student uses AI
Prompt_Engineering_Skill	Self-assessed proficiency in crafting effective prompts
Tool_Diversity	Number of distinct AI tools used
Perceived_AI_Dependency	Student's self-rated dependency on AI tools
Paid_Subscription	Whether the student has a paid AI subscription

The target variable is 'Burnout_Risk_Level'. This is the assessed burnout risk.

We are not interested in any of the other features.

```
In [4]: #Eliminate all other features
feature_names = [
```

```

    'Weekly_GenAI_Hours',
    'Primary_Use_Case',
    'Prompt_Engineering_Skill',
    'Tool_Diversity',
    'Paid_Subscription',
    'Perceived_AI_Dependency'
]

df = df.drop([
    column for column in df.columns
    if column not in feature_names
    and column != 'Burnout_Risk_Level'
],
axis= 1
)

```

Analytical Approach and Model Development

We analyze the dataset. Then we train several models on this dataset.

Structure

We observe 50k rows, none of which have any missing values. However, there are some string-types and a boolean that will need to be encoded.

In [5]: `df.info()`

```

<class 'pandas.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 7 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Weekly_GenAI_Hours                    50000 non-null  float64
1   Primary_Use_Case                      50000 non-null  str
2   Prompt_Engineering_Skill              50000 non-null  str
3   Tool_Diversity                        50000 non-null  int64
4   Paid_Subscription                     50000 non-null  bool
5   Perceived_AI_Dependency               50000 non-null  int64
6   Burnout_Risk_Level                    50000 non-null  str
dtypes: bool(1), float64(1), int64(2), str(3)
memory usage: 2.3 MB

```

The following manipulations are required:

- The target Burnout_Risk_Level needs to be label-encoded
- Primary_Use_Case needs to be one-hot encoded
- Prompt_Engineering_Skill needs to ordinal-encoded

- Paid_Subscription needs to be converted to an integer

```
In [6]: ## label-encode the target
burnout_encoder = LabelEncoder()
df['Burnout_Risk_Level'] = burnout_encoder.fit_transform(df['Burnout_Risk_Level'])
```

```
In [7]: ## Convert Paid_Subscription to int
df['Paid_Subscription'] = df['Paid_Subscription'].astype(int)
```

```
In [8]: ## Label encode prompt-engineering skills
df['Prompt_Engineering_Skill'].unique()
```

```
Out[8]: <StringArray>
['Beginner', 'Advanced', 'Intermediate']
Length: 3, dtype: str
```

```
In [9]: prompt_engineering_encoder = OrdinalEncoder()

df['Prompt_Engineering_Skill'] = prompt_engineering_encoder.fit_transform(df[['Prompt_Engineering_Skill']])
```

```
In [10]: ## One-hot-encode Primary_Use_Case using pandas get_dummies
df = pd.get_dummies(df, drop_first=True, dtype= int)
```

We observe that every column is now numerical

```
In [11]: df.info()

<class 'pandas.DataFrame'>
RangeIndex: 50000 entries, 0 to 49999
Data columns (total 10 columns):
#   Column                                                                 Non-Null Count  Dtype  
---  -
0   Weekly_GenAI_Hours                                                    50000 non-null float64  
1   Prompt_Engineering_Skill                                              50000 non-null float64  
2   Tool_Diversity                                                        50000 non-null int64  
3   Paid_Subscription                                                     50000 non-null int64  
4   Perceived_AI_Dependency                                               50000 non-null int64  
5   Burnout_Risk_Level                                                    50000 non-null int64  
6   Primary_Use_Case_Debugging/Troubleshooting                         50000 non-null int64  
7   Primary_Use_Case_Direct_Answer_Generation                         50000 non-null int64  
8   Primary_Use_Case_Ideation                                             50000 non-null int64  
9   Primary_Use_Case_Summarizing_Reading                               50000 non-null int64  
dtypes: float64(2), int64(8)
memory usage: 3.8 MB
```

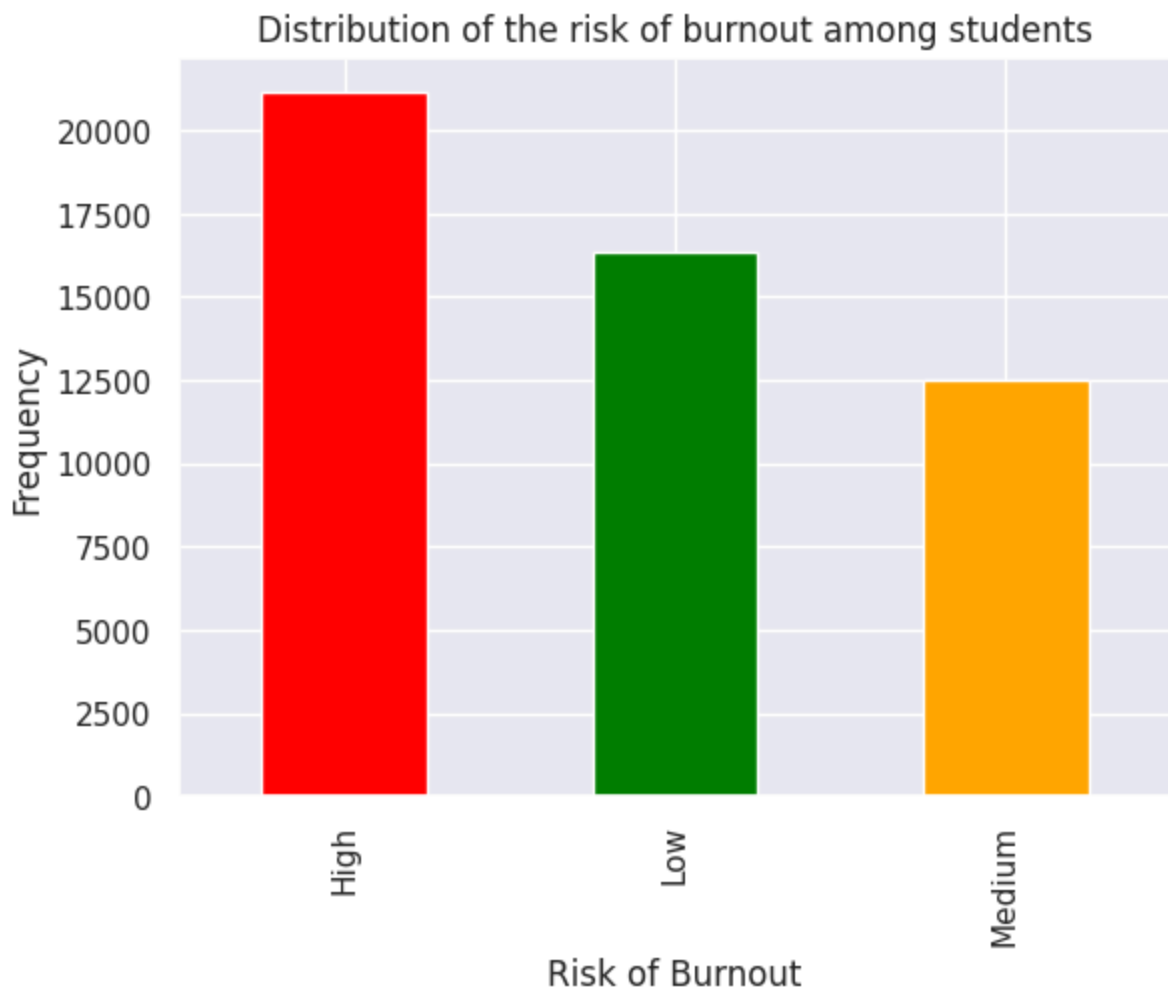
```
In [12]: df.describe().T
```

Out[12]:		count	mean	std	min	2
	Weekly_GenAI_Hours	50000.0	8.427752	8.269490	0.0	2
	Prompt_Engineering_Skill	50000.0	1.077740	0.789980	0.0	(
	Tool_Diversity	50000.0	2.800260	1.188020	1.0	2
	Paid_Subscription	50000.0	0.423080	0.494053	0.0	(
	Perceived_AI_Dependency	50000.0	3.505360	1.820812	1.0	2
	Burnout_Risk_Level	50000.0	1.173140	0.801658	0.0	2
	Primary_Use_Case_Debugging/ Troubleshooting	50000.0	0.245900	0.430624	0.0	(
	Primary_Use_Case_Direct_Answer_Generation	50000.0	0.126800	0.332752	0.0	(
	Primary_Use_Case_Ideation	50000.0	0.214420	0.410423	0.0	(
	Primary_Use_Case_Summarizing_Reading	50000.0	0.172660	0.377957	0.0	(

Data Analysis

We start by plotting the distribution of the target. Observe that the dataset is slightly skewed, but it's not drastic enough to warrant extensive correction.

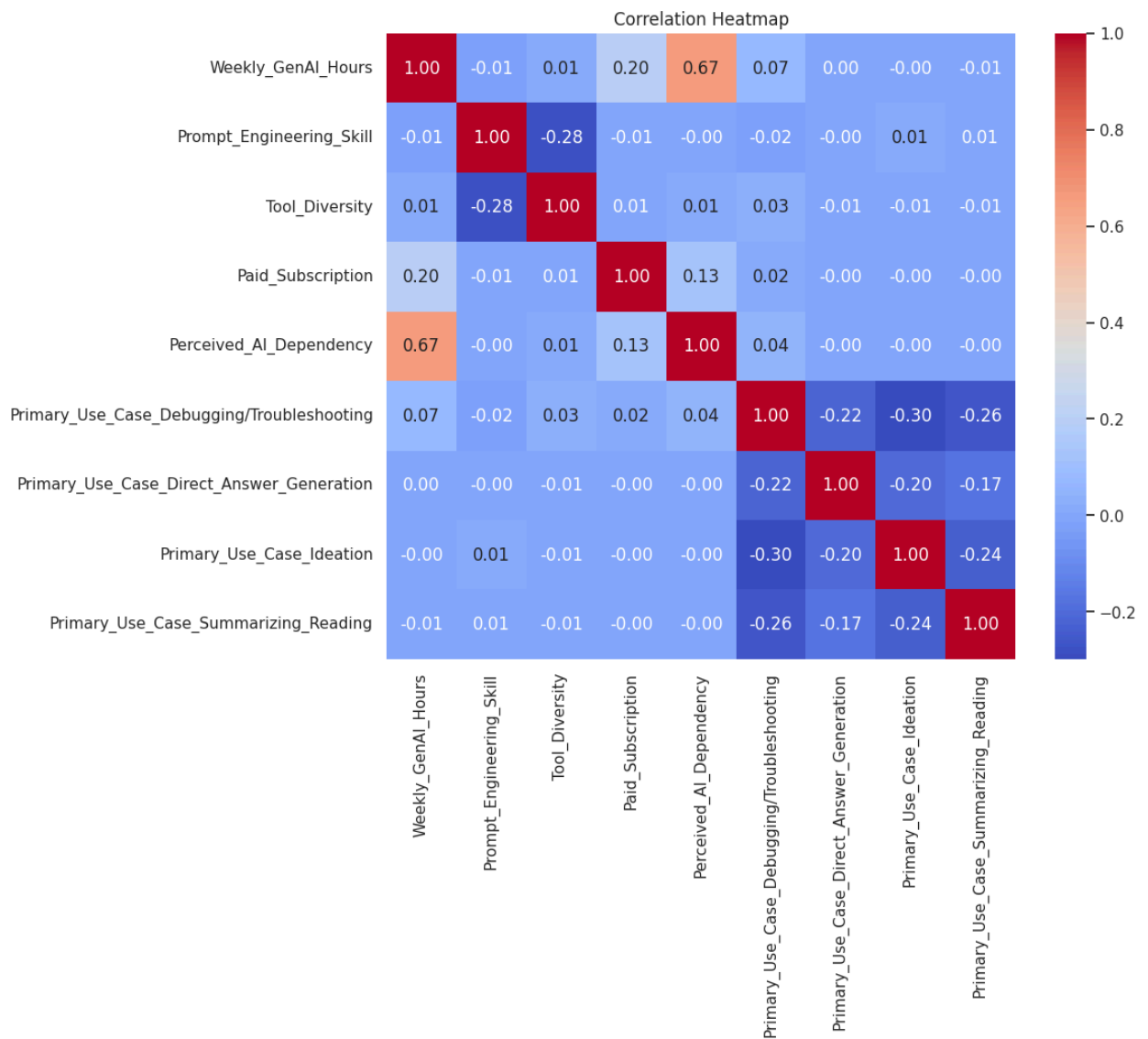
```
In [13]: df['Burnout_Risk_Level'].value_counts().plot.bar(color= ['red', 'green', 'orange'])
plt.xlabel('Risk of Burnout')
plt.ylabel('Frequency')
plt.title('Distribution of the risk of burnout among students')
plt.xticks([0, 1, 2], burnout_encoder.inverse_transform([0, 1, 2]));
```

We proceed to study feature-correlation.

We observe no features are correlated so much that we want to omit them. Of note is that perceived AI dependency is highly related with AI usage.

```
In [14]: plt.figure(figsize=(10, 8))  
corr = df.drop('Burnout_Risk_Level', axis=1).corr()  
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f')  
plt.title('Correlation Heatmap')  
plt.show()
```



Splitting the Data

We use `StratifiedShuffleSplit` to preserve the distribution presented above.

```
In [15]: X = df.drop('Burnout_Risk_Level', axis=1)
y = df['Burnout_Risk_Level']

sss = StratifiedShuffleSplit(test_size= 0.2, random_state=0)
train_idx, test_idx = next(sss.split(X, y))

X_train = X.iloc[train_idx]
X_test = X.iloc[test_idx]
y_train = y.iloc[train_idx]
y_test = y.iloc[test_idx]
```

Fitting models

We fit different types of models, and compare them afterwards.

Decision Tree Classifier

We first try to fit a decision tree classifier, to see if we are able to overfit the data.

It's often easier to regularize an overfit model than to start with regularizing.

Note that we will be using accuracy as our metric of choice because this is fine with the dataset being this balanced.

```
In [16]: dtc = DecisionTreeClassifier()
dtc.fit(X_train, y_train)

y_pred = dtc.predict(X_test)

print('Training accuracy:', accuracy_score(y_train, dtc.predict(X_train)))
print('Testing accuracy:', accuracy_score(y_test, y_pred))
```

Training accuracy: 0.98315

Testing accuracy: 0.4162

Because the train accuracy is way higher than the test accuracy, this model is clearly overfit. Now let's regularize the decision tree classifier to obtain better test results.

```
In [17]: dtc_param_grid = {
    "max_depth": range(1, 21),
    "max_features": range(1, 10)
}

dtc_search = GridSearchCV(
    estimator= dtc,
    param_grid = dtc_param_grid,
    n_jobs= -1
)

dtc_search.fit(X_train, y_train)
best_dtc = dtc_search.best_estimator_

y_pred = best_dtc.predict(X_test)

print('Training accuracy:', accuracy_score(y_train, best_dtc.predict(X_train)))
print('Testing accuracy:', accuracy_score(y_test, y_pred))
```

Training accuracy: 0.508125

Testing accuracy: 0.5072

```
In [18]: #We verify that we choose the right values for hyperparameter optimization  
dtc_search.best_params_
```

```
Out[18]: {'max_depth': 4, 'max_features': 9}
```

We find an accuracy of approximately 0.5. This is better than guessing, but not extremely strong.

Logistic Regression

Let's try logistic regression.

```
In [19]: lr_steps = [  
    ('ss', StandardScaler()),  
    ('lr', LogisticRegression())  
]  
  
lr_pipe = Pipeline(lr_steps)  
lr_pipe.fit(X_train, y_train)  
  
y_pred = lr_pipe.predict(X_test)  
  
print('Training accuracy:', accuracy_score(y_train, lr_pipe.predict(X_train)))  
print('Testing accuracy:', accuracy_score(y_test, y_pred))
```

Training accuracy: 0.507825

Testing accuracy: 0.508

This model isn't powerful enough in its standard form. Let's try adding polynomial features and changing the amount of regularization.

```
In [20]: lr_steps = [  
    ('pf', PolynomialFeatures(include_bias= False)),  
    ('ss', StandardScaler()),  
    ('lr', LogisticRegression(max_iter= 1000))  
]  
lr_pipe = Pipeline(lr_steps)  
  
lr_param_grid = {  
    'pf__degree': range(1, 4),  
    'lr__C' : [0.01, 0.1, 1.0, 10.0, 100.0]  
}  
  
lr_search = GridSearchCV(  
    estimator= lr_pipe,  
    param_grid= lr_param_grid,  
    n_jobs= -1  
)  
  
lr_search.fit(X_train, y_train)  
best_lr = lr_search.best_estimator_
```

```
y_pred = best_lr.predict(X_test)

print('Training accuracy:', accuracy_score(y_train, best_lr.predict(X_train)))
print('Testing accuracy:', accuracy_score(y_test, y_pred))
```

Training accuracy: 0.5076
Testing accuracy: 0.5079

```
In [21]: #We verify that we choose the right values for hyperparameter optimization
lr_search.best_params_
```

```
Out[21]: {'lr__C': 0.01, 'pf__degree': 1}
```

Adding polynomial features did pretty much nothing. The results after adding polynomial features and tuning the regularization are pretty much identical.

We find an accuracy of approximately 0.5. This is better than guessing, but not extremely strong.

K nearest Neighbours

We don't have a lot of features, so K nearest is a good model to try as we are not suffering from the curse of dimensionality.

```
In [22]: knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)

print('Training accuracy:', accuracy_score(y_train, knn.predict(X_train)))
print('Testing accuracy:', accuracy_score(y_test, y_pred))
```

Training accuracy: 0.689825
Testing accuracy: 0.4223

This is better than the logistic regression, so maybe we can find a better model here.

```
In [23]: knn_param_grid = {
          'n_neighbors' : range(1, 300, 4)
        }

knn_search = GridSearchCV(
    estimator= knn,
    param_grid = knn_param_grid,
    n_jobs= -1
)

knn_search.fit(X_train, y_train)
```

```

best_knn = knn_search.best_estimator_

y_pred = best_knn.predict(X_test)

print('Training accuracy:', accuracy_score(y_train, best_knn.predict(X_train)))
print('Testing accuracy:', accuracy_score(y_test, y_pred))

```

Training accuracy: 0.508975

Testing accuracy: 0.5117

In [24]: *#We verify that we choose the right values for hyperparameter optimization*
knn_search.best_params_

Out[24]: {'n_neighbors': 293}

We seem to get an accuracy of 0.5 again. Still better than guessing.

Support Vector Machine

The last model covered in this course is SVM. We try that here.

In [25]:

```

svc = SVC()
svc.fit(X_train, y_train)

y_pred = svc.predict(X_test)

print('Training accuracy:', accuracy_score(y_train, svc.predict(X_train)))
print('Testing accuracy:', accuracy_score(y_test, y_pred))

```

Training accuracy: 0.507325

Testing accuracy: 0.5094

We immediately obtain equal accuracy without any hyperparameter tuning. We still try our best to improve, but it's unlikely it will work.

In [26]:

```

svc_param_grid = {
    "C" : [0.1, 1, 10]
}

svc_search = GridSearchCV(
    estimator= svc,
    param_grid= svc_param_grid,
    n_jobs= -1
)

svc_search.fit(X_train, y_train)
best_svc = svc_search.best_estimator_

y_pred = best_svc.predict(X_test)

print('Training accuracy:', accuracy_score(y_train, best_svc.predict(X_train)))

```

```
print('Testing accuracy:', accuracy_score(y_test, y_pred))
```

Training accuracy: 0.5087

Testing accuracy: 0.5085

We obtain an accuracy of 0.5, which is identical to that of our models.

Stacking all the models together

Let's see what happens when we stack all the models together.

```
In [27]: estimators = [  
          ('dtc', best_dtc),  
          ('lr', best_lr),  
          ('knn', best_knn),  
          ('svc', best_svc)  
        ]  
  
        stack = VotingClassifier(  
            estimators= estimators  
        )  
        stack.fit(X_train, y_train)  
  
        y_pred = stack.predict(X_test)  
  
        print('Training accuracy:', accuracy_score(y_train, stack.predict(X_train)))  
        print('Testing accuracy:', accuracy_score(y_test, y_pred))
```

Training accuracy: 0.50945

Testing accuracy: 0.5079

It seems that stacking all models still yields an accuracy of 0.5.

Comparing different models

We give different error metrics of all models, to see if a certain model might be better to predict who has a high risk of burn-out than others.

```
In [28]: performance_overview = pd.DataFrame()  
  
        models = [  
            ('dtc', best_dtc),  
            ('lr', best_lr),  
            ('knn', best_knn),  
            ('svc', best_svc),  
            ('stack', stack)  
        ]  
  
        for model in models:
```

```

y_pred = model[1].predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
precision, recall, f1, _ = precision_recall_fscore_support(y_test, y_pred)

score_overview = {
    'accuracy': accuracy,
    'precision_high_risk': precision[0],
    'recall_high_risk': recall[0],
    'f1_high_risk': f1[0]
}

performance_overview[model[0]] = score_overview

performance_overview

```

Out[28]:

	dtc	lr	knn	svc	stack
accuracy	0.507200	0.507900	0.511700	0.508500	0.507900
precision_high_risk	0.639744	0.650726	0.648214	0.656250	0.646989
recall_high_risk	0.440929	0.430517	0.436123	0.420505	0.434521
f1_high_risk	0.522048	0.518197	0.521427	0.512570	0.519885

The models perform very similar, but some useful findings:

- logistic regression and support vector machine have the highest precision.
- knn and dtc have the highest recall.
- the stack is somewhere in between.

Conclusions

All models have an accuracy of around 0.5. Given that there are 3 classes, this is significantly better than guessing!

Hence, we conclude that there is a significant correlation between AI usage among students and risk of burnouts.

Use of these models

If your goal is to identify as many students as possible with a high risk of burnout, I'd recommend the models Decision Tree Classifier or Support Vector Machine. These models find 45% of all students with a high risk of burnout.

If your goal is to have measures taken to help people with a high risk of burnout be as likely as possible to help someone who actually has a high risk, I'd recommend the models Logistic Regression or K Nearest Neighbors. These models are right 66% percent of the time when identifying a student as 'high risk of burnout'.

Limitations and Flaws

The models in this notebook are trained only on data of AI-usage of students. This was done to study if a relation between AI-usage and burnout can be found. It is evident that such a relationship exists.

However, these models are not great for pinpointing students with a high risk of burnout. Obviously, there are more factors to burnout than AI-usage.

Improvements and Next steps

One way to find a better way of pinpointing these students is train the models on the entire Kaggle dataset. This would yield stronger models, that are more useful in the real world. To achieve this, one would retrain all the models mentioned above on the entire dataset, and find which model performs best.

Another improvement that could be is to properly interpret these models. By means of our error-measuring before, we have done some interpretation already. However, more could be done. One possible way to interpret these models is to use LIME. This would be my starting point.